

Extensão CPU/GPU do HFlow: as premissas de desempenho, confrontadas com a medida

Relatório técnico

Setembro de 2026

Resumo

O projeto *Extensão da Arquitetura HFlow para Execução Acelerada em CPU/GPU em Malhas Hierárquicas* propõe três coisas: um núcleo de software compartilhado entre HiGFlow e MFSim-NG, o t8code como opção de particionamento, e execução em GPU no *Solver* e no operador mais custoso de *Discretization*.

Este documento confronta as *premissas de desempenho* desse projeto com medidas feitas no código. **Não é uma avaliação do projeto**, cujo núcleo é arquitetural e independe do que segue: unificar duas implementações divergentes tem valor próprio. É a contribuição de uma medida para a *ordem* das extensões.

Três achados. O remalhamento custa entre 0,06 % e 0,10 % do tempo, de modo que a troca do mecanismo de particionamento otimiza, na escala medida, algo negligenciável. A premissa de que o ajuste de mínimos quadrados do GFD é o operador mais caro **nunca foi medida**, e não há instrumento no código que separe montagem de resolução. E os gargalos estruturais que existem estão do lado da interface que o *princípio de não invasividade* promete não tocar — uma tensão a decidir, não um defeito a corrigir.

Sumário

1	As medidas que esta análise usa	1
1.1	Custo do remalhamento e escalabilidade	2
1.2	Iterações do solver linear	2
2	As três premissas de desempenho	2
3	O que foi medido	2
4	Três consequências	3
5	O que isto sugere para a ordem do trabalho	3
6	O alcance desta seção, e os seus limites	4

1 As medidas que esta análise usa

Duas campanhas de medida no HiGFlow sustentam o que segue. Ambas foram feitas sobre o caso de bolha ascendente bifásica (*benchmark* de Hysing), com cerca de 9 400 células — e essa pequenez é a principal limitação deste documento, dita aqui e retomada no fim.

1.1 Custo do remalhamento e escalabilidade

O critério de refino e a reconstrução do domínio foram cronometrados **separados do resto**, com barreira antes de cada marca, em $n_p = 1, 2, 4$ e 8 :

n_p	parede	aceleração	eficiência	critério	reconstrução	remalha/total
1	998,8 s	1,00×	100%	0,0040 s	0,1621 s	0,083%
2	1219,0 s	0,82×	41%	0,0042 s	0,1506 s	0,064%
4	864,9 s	1,15×	29%	0,0058 s	0,1345 s	0,081%
8	568,7 s	1,76×	22%	0,0025 s	0,1085 s	0,098%

1.2 Iterações do solver linear

n_p	pressão	vs. base	velocidade
1	110,1	—	4,5
2	116,2	+5,5%	6,0
4	117,7	+6,8%	6,0
8	122,5	+11,2%	6,1

A pressão domina o custo linear com folga — 110 iterações contra 4 a 6 da velocidade —, como se espera da equação de Poisson da projeção. O crescimento de 11 % com o número de processos é a degradação do preconditionador **bjacobi** ao ser dividido em mais blocos, e responde por cerca de 13 % da aceleração perdida.

Há um projeto de pesquisa em preparação — *Extensão da Arquitetura HFlow para Execução Acelerada em CPU/GPU em Malhas Hierárquicas* — que propõe três coisas: um núcleo de software compartilhado entre HiGFlow e MFSim-NG, o t8code como nova opção de particionamento, e uma camada de execução em GPU no *Solver* e no operador mais custoso de *Discretization*.

Esta seção confronta as premissas de desempenho desse projeto com o que foi medido no código. **Não é uma avaliação do projeto**, cujo núcleo é arquitetural e independe do que segue; é a contribuição de uma medida para a *ordem* das extensões.

2 As três premissas de desempenho

premissa do projeto	estado	o que a medida diz
o MTree limita células e balanceamento	não verificada	o remalhamento custa 0,06 %–0,10 %
o ajuste MLS do GFD é o operador mais caro	não medida	não há cronômetro separando montagem
a extensão é não invasiva (não toca a aplicação)	verdadeira	e é por isso que ela não alcança os

3 O que foi medido

grandeza	valor	procedência
fração do remalhamento no tempo	0,06 %–0,10 %	medido
eficiência paralela, $n_p = 2$	41 %	medido
eficiência paralela, $n_p = 8$	22 %	medido
iterações do KSP na pressão	110 a 122 (n_p de 1 a 8)	medido
iterações do KSP na velocidade	4 a 6	medido
custo de montagem vs. resolução	não medido	sem instrumento

4 Três consequências

1. O t8code otimizaria, hoje, algo que custa 0,1 %. Isso **não refuta** a extensão: o projeto mira malhas muito maiores que a medida aqui, e nesse regime o equilíbrio muda. O que a medida estabelece é que o ganho é *invisível na escala atual*, e portanto que a justificativa não pode ser verificada sem antes levar um caso ao tamanho-alvo.

2. A premissa da GPU não está medida, e a única evidência disponível aponta para o outro lado. O projeto identifica o ajuste de mínimos quadrados do GFD como o operador mais caro. Montagem e resolução **nunca foram cronometradas separadamente** — não há instrumento no código que as separe. O que há é a contagem de iterações: a pressão consome 110 iterações de KSP por passo contra 4 a 6 da velocidade, o que sugere peso na *resolução*.

Pode ser que o MLS domine mesmo assim — montagem cara e poucas iterações não se excluem. O ponto é que a priorização entre *Solver* e *Discretization* hoje repousa sobre asserção, e que separá-las custa um cronômetro.

3. E aqui está o ponto mais delicado. Os três gargalos estruturais identificados na leitura do código são:

1. uma única árvore `.amr` materializada *inteira num processo* antes do particionamento;
2. o domínio **reconstruído do zero** a cada remalhamento, em vez de adaptado no lugar;
3. o critério de refino passando por **arquivo escrito serialmente**, lido por todos.

Nenhum deles está no *backend* de particionamento. Estão na camada da aplicação e do núcleo — do lado da interface que o *princípio de não invasividade* do projeto promete não tocar.

Trocar MTree por t8code atrás da mesma interface de consulta topológica melhora a qualidade da partição de uma malha que continua sendo montada globalmente num processo. **O critério de projeto que protege a arquitetura é, nesse sentido, o que impede a extensão de alcançar os gargalos** — e isso é uma tensão real a decidir, não um defeito a corrigir: não invasividade e realocação de gargalo são objetivos que competem.

5 O que isto sugere para a ordem do trabalho

O próprio projeto prevê, na fase F1, um *levantamento de baseline*: *benchmarks de desempenho e de escalabilidade da arquitetura atual (CPU, MTree)*. A sugestão que decorre desta seção é tratá-lo como **decisório, e não preliminar**: fazer a *baseline* no tamanho-alvo antes de fixar a ordem das extensões, porque ela pode relocar o gargalo.

Três medidas resolveriam as dúvidas acima, e as três são baratas:

- **montagem × resolução**, cronometradas em separado — decide se a GPU começa pelo *Solver* ou pelo *Discretization*;
- **escalabilidade forte num caso grande** — o medido aqui tem cerca de 9 400 células, e a 22 % de eficiência a conclusão é sobre o *tamanho*, não sobre a arquitetura;
- **memória por processo ao aumentar n_p** — é o teste direto do primeiro gargalo, e o único que diria se a materialização global morde antes do resto.

6 O alcance desta seção, e os seus limites

Convém delimitar, porque a seção faz afirmações sobre um projeto a partir de medidas estreitas:

- **um único caso, e pequeno:** bolha ascendente bifásica, cerca de 9 400 células. Nada aqui se estende automaticamente a malhas grandes, que são o alvo do projeto;
- **um único software:** tudo foi medido no HiGFlow. O MFSim-NG usa FV e não GFD, e a premissa sobre o operador mais caro pode ser verdadeira lá e falsa aqui, ou o contrário;
- **a arquitetura não foi avaliada.** O núcleo compartilhado é a parte principal do projeto, e nada nesta seção o toca — unificar duas implementações divergentes tem valor independente de desempenho;
- **os três gargalos são fatos de código; que eles *dominem* é inferência,** e a medida da inventário do sistema mostrou que inferências desse tipo já caíram uma vez neste trabalho.

Referências